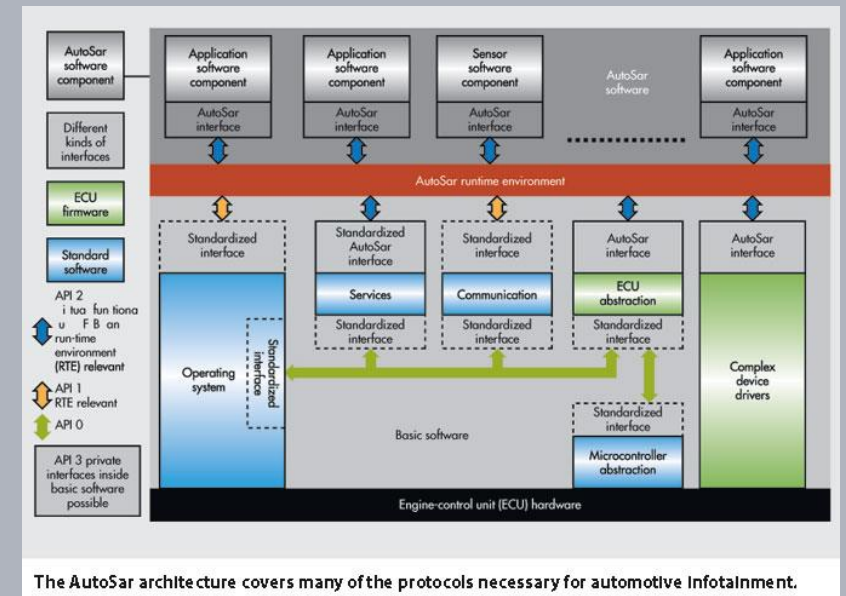


Advanced Microcontroller Architectures and Embedded OS

Statemachine Basics

Prof. Dr.-Ing. Peter Fromm



The AutoSar architecture covers many of the protocols necessary for automotive Infotainment.

Content

- Motivation State Machines
- State Machine Basics
- Switch-Case Pattern
- Example MP3 Player

Example: MP3 Player



Statemachine Definitions

Definition

A finite state machine is an abstract machine that defines a finite set of conditions of existence (called “states”), a set of behaviors or actions performed in each of those states, and a set of events that cause changes in states according to a finite and well-defined rule set.

Bruce Powel Douglass

<http://www.embedded.com/1999/9901/9901feat1.htm>

When to use state machines?

- Objects with behavior, that is significant different depending on internal object condition (state, operation mode etc.)
- The state is stored in an internal state variable (object attribute). It is not accessible from outside. State changes **occur only** as (possible) side-effect of event processing.
- Major use case for state machines: reactive systems, e.g.
 - All devices with direct user interactions (MP3 player, mobile phone,...)
 - All devices communicating to other devices via networks (Routers, Printers,....)
 - ...

Systems having states operate differently upon the same input, depending on their state.

How to localize states?

- Many software systems e.g. in automotive possess a system state (which typically distinguishes STATE_Standby, STATE_Slow, STATE_StandbyRun, STATE_Active, STATE_Boardtest and other operation modes)
- Modules may have in addition their own state machines.
 - HMI components like widgets typically possess a state machine reflecting its display state (REGISTERED, PREPARED, VISIBLE,...).
 - CAN or other drivers show behavior like BUSY, FREE, TRANSMITTING etc.

Every complex if structure or nested switch statement in code is an indicator for a state machine!

Systems without states

- Objects without behavior, i.e. without significant difference of offered functionality or service can't be viewed as active object
- No triggering events can be identified
- Fire and forget instances
- Global or static variables **without impact to behavior** (function/statement execution) can't be viewed as state of an active object

Basic notations

State

- operational mode of an object (or a system)
- each state should have a descriptive name
- all possible states must be enumerated
- the object has a current state (stored in a private variable)

Event

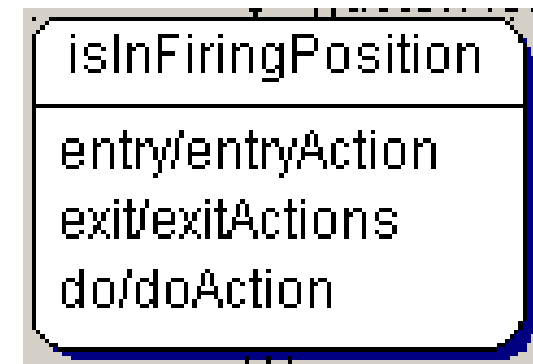
- externally caused triggers for actions (and possible state changes)
- an event may be time-related (cyclic, timeout, specific time)
- or sporadic (e.g. key event, message from other objects)

Transition

- state change of an object
- typically associated with transition action, i.e. method calls
- may be subject to a guard condition (if the guard evaluates to false at runtime the transition is not taken)

States

- A state represents an internal condition of the software
- Depending on the state, the software reacts differently on events
- States may be extended by action expressions
- Action Expressions
 - entry / Action Expression → executed upon entering the state from any other state
 - exit / Action Expression → executed upon leaving the state to any other state
 - do / Action Expression → executed while the software is in this state and any event occurred (including events which are not relevant for this state)

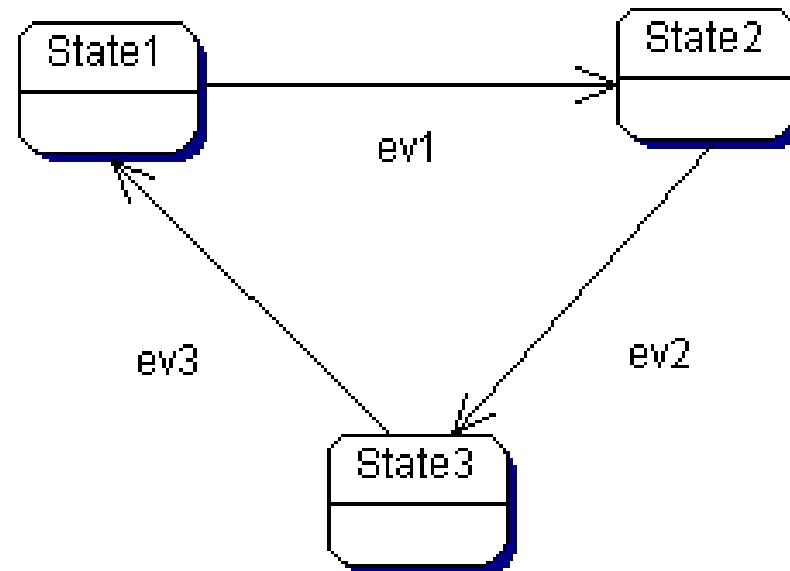


Events

- Events are signals which are sent to the software. They can be considered as software interrupts
- Events can be caused by
 - User Input
 - Sensors
 - Communication Signals
 - Timeouts (cyclic, non-cyclic)
 - Application logic
- In Autosar, events are technically fired
 - By an Alarm
 - Or by a SetEvent
- Events can be
 - considered → if the statemachine is in a state which reacts on the event
 - ignored → if the statemachine is in a state which does not react on the event

Events

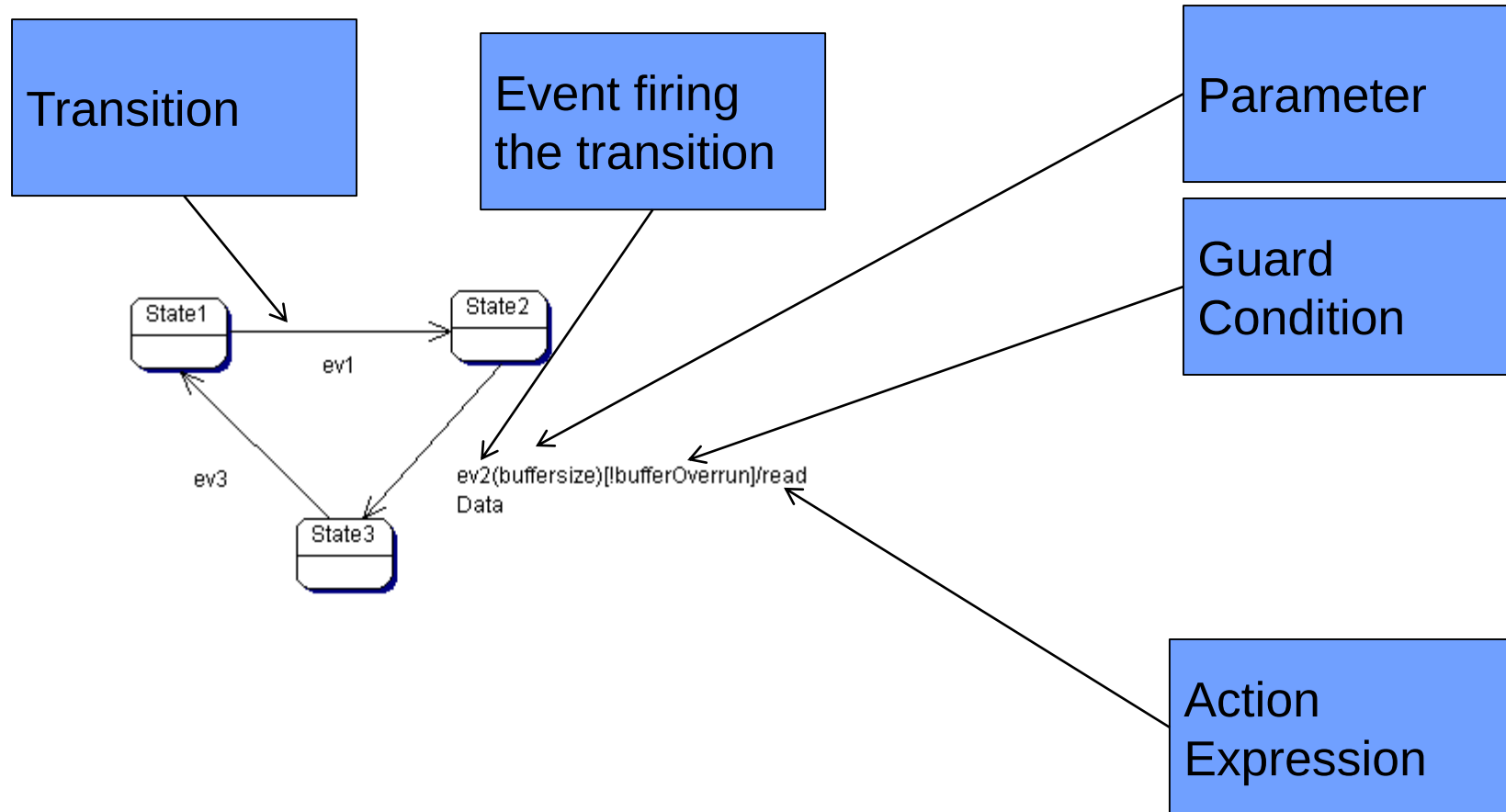
- System is in State1
- ev2 is fired → ev2 is ignored, no transition is executed
- ev1 is fired → ev1 is considered and System changes to State2



Transitions

- Transitions connect two states
- Transitions are triggered by events
- Transitions may be optionally extended by
 - ActionExpressions → which are executed when the event is considered
 - Parameters → which are passed to the event handler
 - Guard Conditions → if the guard condition is true, the event is considered, otherwise it is ignored
- Syntax
 - evName / ActionExpression
 - evName[Guard Condition]/Action Expression
 - evName(Parameter) [Guard Condition]/Action Expression

Transitions



Action Expressions and Guard Conditions

Action Expressions and Guard Conditions are typically implemented as small functions (increases maintainability).

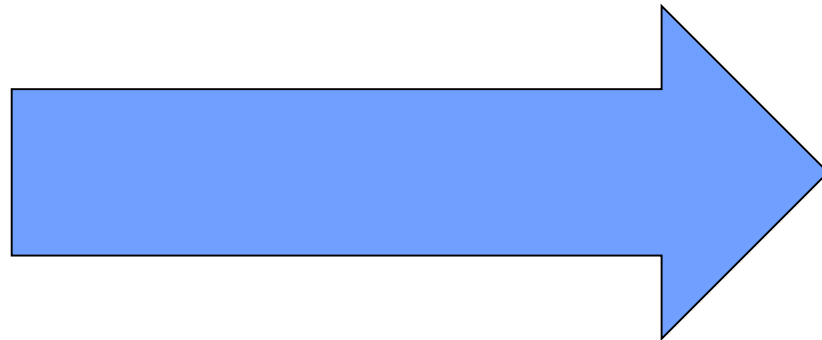
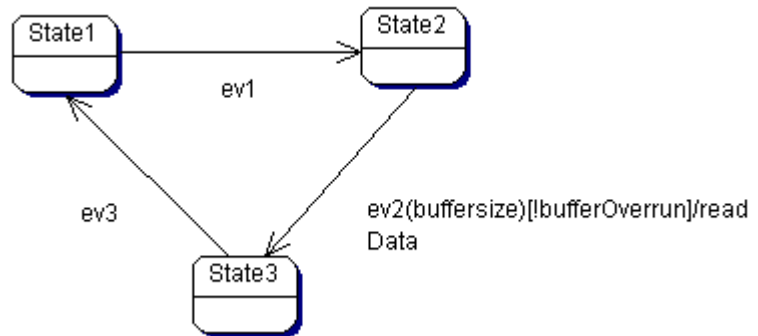
If performance is an issue, these can be inlined.

```
void DeactivateActors()
{
    rubberSpeed = 0;
    rubberPosition = 0;
    gunSpeed = 0;
    gunAngle = 0;
}

void SetMoveGunUp()
{
    gunSpeed = GUNSPEED;
}

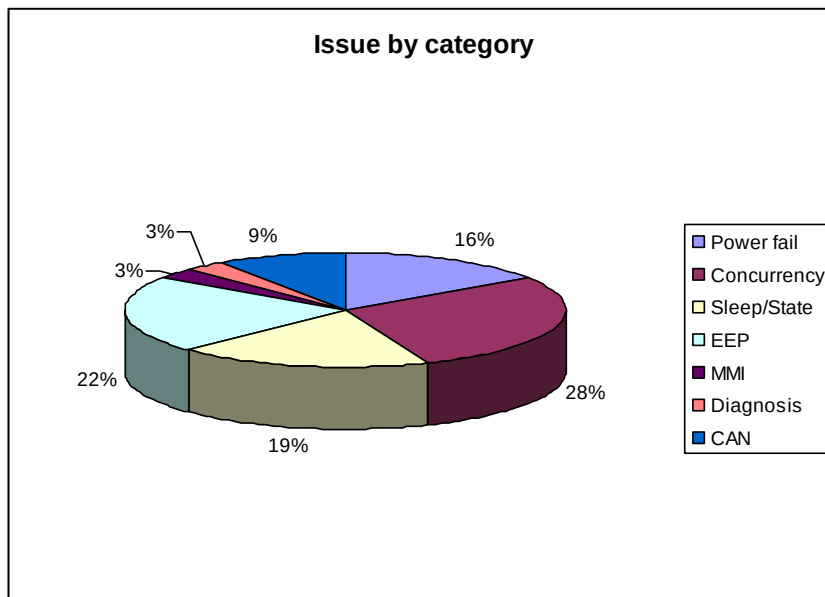
bool BufferIsFree()
{
    return (currentPointer <= MAXBUFFER);
}
```

Design to Code



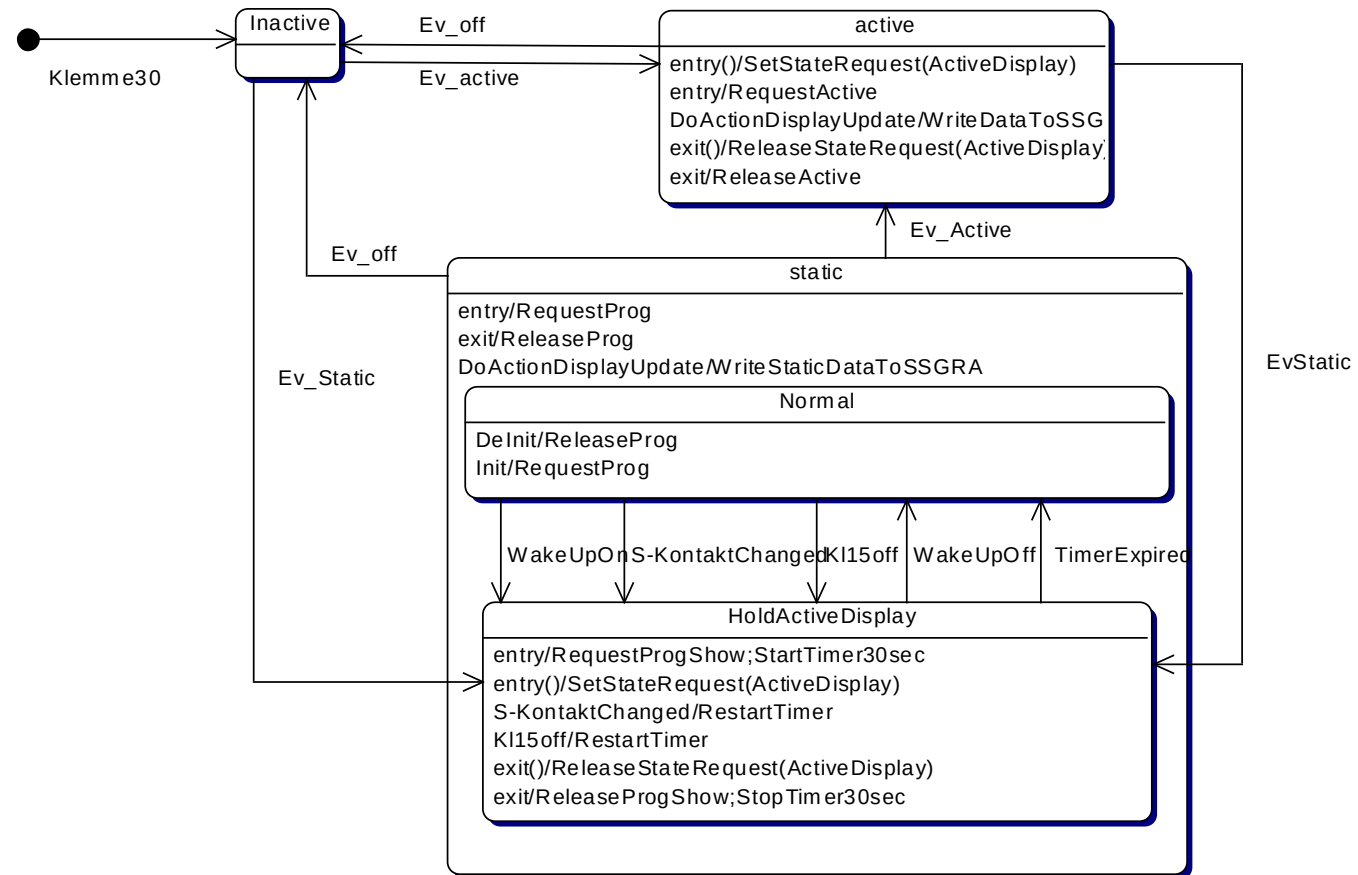
Field Problems in Automotive

- Power fail (16%) in relation with other modules (Initialization, EEPROM read/write (22%), CAN (9%), detection of external devices)
- Concurrency (28%) (Data inconsistency, dead locks, lacking thread safety)
- Errors related to Standby and Sleep (19%) (caused by inconsistent system state requests)
- Complex MMI scenarios (Display and widget states)



Often caused by wrongly designed or implemented state machines!

Design Grafic Driver



Code is spread over several functions

```
214 /* set main or sub state */
215 void AUHMI_vSetStates (void)
216 {
217     ReceiveMessage (MSG_AUHSTZState, &stAUHState);
218     if (stAUHState.biActiveStateHeat || stAUHState.biActiveStateVent)
219     {
220         AUHMI_vStateHandler (AUHMI_nEvActive);
221     }
222     else
223     {
224         if (stAUHState.biProgStateVent || stAUHState.biProgStateHeat)
225         {
226             AUHMI_vStateHandler (AUHMI_nEvStatic);
227         }
228         else
229         {
230             AUHMI_vStateHandler (AUHMI_nEvOff);
231         }
232     }
233     if (stAUHState.biWeckzeit != AUHMI_nEvOff)
234     {
235         if (stAUHState.biWeckzeit)
236         {
237             AUHMI_vStateHandler (AUHMI_nEvWakeUp);
238         }
239         else
240         {
241             AUHMI_vStateHandler (AUHMI_nEvWakeUp);
242         }
243     }
244     AUHMI__stState.biWakeUpOld = stAUHState.biWakeUp;
245 }
246 /* AUHMI_vStateHandler */

247 void AUHMI_vStateHandler (AUHMI_tenStateEvents enStateEvent)
248 {
249     switch (AUHMI__stState.biMainState)
250     {
251     case AUHMI_nInactive:
252         if (enStateEvent == AUHMI_nEvStatic)
253         {
254             AUHMI__stState.biMainState = AUHMI_nStatic;
255             AUHMI__stState.biSubState = AUHMI_nHoldDisplay;
256             MMI_u8Request(MMI_nApAuxHeatProg, MMI_nLrAuxHeatProg, MMI_u8Request(MMI_nApAuxHeatProgShow, MMI_nLrAuxHeatProgShow);
257             CSHDL_enReqHd1SetRequest(CSHDL_nReqSMRES_ActiveDisplayByAUHMI);
258             TIMER_vStart(TIMER_nAUHMI_ShowProgDelay, AUHMI_nTimerVal);
259         }
260         if (enStateEvent == AUHMI_nEvActive)
261         {
262             AUHMI__stState.biMainState = AUHMI_nActive;
263             CSHDL_enReqHd1SetRequest(CSHDL_nReqSMRES_ActiveDisplayByAUHMI);
264             MMI_u8Request(MMI_nApAuxHeatActive, MMI_nLrAuxHeatActive, MMI_u8Request(MMI_nApAuxHeatActive, MMI_nLrAuxHeatActive);
265             break;
266         }
267     case AUHMI_nActive:
268         if (enStateEvent == AUHMI_nEvOff)
269         {
270             CSHDL_vReqHd1ClearRequest(CSHDL_nReqSMRES_ActiveDisplayByAUHMI);
271             MMI_u8Release(MMI_nApAuxHeatActive, MMI_nLrAuxHeatActive);
272             AUHMI__stState.biMainState = AUHMI_nInactive;
273         }
274         if (enStateEvent == AUHMI_nEvStatic)
275         {
276             CSHDL_vReqHd1ClearRequest(CSHDL_nReqSMRES_ActiveDisplayByAUHMI);
277             MMI_u8Release(MMI_nApAuxHeatActive, MMI_nLrAuxHeatActive);
278             AUHMI__stState.biMainState = AUHMI_nStatic;
279             AUHMI__stState.biSubState = AUHMI_nHoldDisplay;
280             MMI_u8Request(MMI_nApAuxHeatProg, MMI_nLrAuxHeatProg, MMI_u8Request(MMI_nApAuxHeatProgShow, MMI_nLrAuxHeatProgShow);
281             CSHDL_enReqHd1SetRequest(CSHDL_nReqSMRES_ActiveDisplayByAUHMI);
282             TIMER_vStart(TIMER_nAUHMI_ShowProgDelay, AUHMI_nTimerVal);
283         }
284         break;
285     case AUHMI_nStatic:
286         if (enStateEvent == AUHMI_nEvOff)
287         {
288             if (AUHMI__stState.biSubState == AUHMI_nHoldDisplay)
289             {
290                 CSHDL_vReqHd1ClearRequest(CSHDL_nReqSMRES_ActiveDisplayByAUHMI);
291                 MMI_u8Release(MMI_nApAuxHeatProgShow, MMI_nLrAuxHeatProgShow);
292                 TIMER_vClear(TIMER_nAUHMI_ShowProgDelay);
293             }
294         }
295     }
296 }

344 /* AUHMI_vWriteDataToSSGRA */
345 /* write data to ssgra */
346 static void AUHMI_vWriteDataToSSGRA(void)
347 {
348     SSGRA_tstData stShowData;
349     /* init structure */
350     SSGRA_vInitData(&stShowData);
351     /* set client id */
352     stShowData.enClient = SSGRA_nu8AuhClient;
353     /* set display mode for ahumi */
354     stShowData.enClientDispMode = SSGRA_nu8StaticData;
355     if (stAUHState.biProgStateVent)
356     {
357         stShowData.unData.stAUH.enVentilationState = SSGRA_nu8_On;
358     }
359     else
360     {
361         stShowData.unData.stAUH.enVentilationState = SSGRA_nu8_Off;
362     }
363     if (stAUHState.biProgStateHeat)
364     {
365         stShowData.unData.stAUH.enHeatingState = SSGRA_nu8_On;
366     }
367     else
368     {
369         stShowData.unData.stAUH.enHeatingState = SSGRA_nu8_Off;
370     }
371     if (stAUHState.biActiveStateVent)
372     {
373         stShowData.enClientDispMode = SSGRA_nu8FlashingData;
374         stShowData.unData.stAUH.enVentilationState = SSGRA_nu8_Flash;
375     }
376     if (stAUHState.biActiveStateHeat)
377     {
378         stShowData.enClientDispMode = SSGRA_nu8FlashingData;
379         stShowData.unData.stAUH.enHeatingState = SSGRA_nu8_Flash;
380     }
381     SSGRA_vWriteData(MMI_nPdSdField3, &stShowData);
382 }
```

Problems

- Events of different levels are mixed
 - KL30 – turning on the Car battery
 - TimeOut, Ev_* - should only occur during operation
- Model is not complete, e. g. guards are present in the implementation, but not in the model
- Part of the logic is in the control function, parts are shifted to the action expressions
- Implementation does event processing at several places (error prone in maintenance mode)

Example: Autogenerated Code

```
54 // Die State-Handler-Funktionen (jede der folgenden Funktionen repraesentiert einen Zustand):
55 // The state handler functions (each of the following functions represents a state):
56 StateFunc AUHMI_SM1_TopState_( AUHMI_SM1* pThis, EcEvent const *pEvent );
57 StateFunc AUHMI_SM1_State003AUHMI_Sinactive_( AUHMI_SM1* pThis, EcEvent const *pEvent ); // [EasyCODE:State3@"D:/m
58 StateFunc AUHMI_SM1_State004AUHMI_Sactive_( AUHMI_SM1* pThis, EcEvent const *pEvent ); // [EasyCODE:State4@"D:/mypr
59 StateFunc AUHMI_SM1_State005AUHMI_Sstatic_( AUHMI_SM1* pThis, EcEvent const *pEvent ); // [EasyCODE:State5@"D:/mypr
60 StateFunc AUHMI_SM1_State006AUHMI_Snormal_( AUHMI_SM1* pThis, EcEvent const *pEvent ); // [EasyCODE:State6@"D:/mypr
61 StateFunc AUHMI_SM1_State007AUHMI_ShouldActiveDisplay_( AUHMI_SM1* pThis, EcEvent const *pEvent ); // [EasyCODE:Sta
62 StateFunc AUHMI_SM1_State002FinalState_( AUHMI_SM1* pThis, EcEvent const *pEvent ); // [EasyCODE:State2@"D:/myproj

218 //! \author EasyCODE Code Generator
219 //! \version 8.0
220 //! \date 11.04.2008, 10:16:23
221 //! This function represents a state.
222 //! It is called when an event occurs and the related state is the currently active state.
223 //! Furthermore it may be called as part of a transition's "exit/entry chain".
224 //! [EasyCODE:State4@"D:/myprojects/StateMachines/testcases/NECV850_20080304/packages/AUHMI/EcModel/AUHMI_SM1.ecs"]
225 StateFunc AUHMI_SM1_State004AUHMI_Sactive_( AUHMI_SM1* pThis, EcEvent const *pEvent )
226 -{
227     if ( eEntryState == pEvent->type ) // is the statemachine entering this state?
228     -{
229         EC_DEBUG_STATE(4, "State004AUHMI_Sactive::Entry");
230     -#ifdef USE_TIME_EVENTS
231         pThis->m_arrTicksElapsedRelative[0/*nesting level*/] = 0;
232     -#endif
233         // Entry-Aktionen dieses Zustands ausfuehren: / perform the entry actions:
234         SetStateRequest(ActiveDisplay);
235         return NULL; // d.h. Event wurde behandelt / i.e. event has been handled
236     }
237     if ( eExitState == pEvent->type ) // Exit-Aktionen dieses Zustands ausfuehren: / perform the exit actions:
238     -{
239         EC_DEBUG_STATE(4, "State004AUHMI_Sactive::Exit");
240         ReleaseStateRequest(ActiveDisplay);
241         return NULL; // d.h. Event wurde behandelt / i.e. event has been handled
242     }
```

State Machine Coding Pattern

There are different solutions how to code a state machine.
The following switch case pattern focuses on

- simple (flat) state machines
- optimized code and runtime
- easy understanding

but has some pitfalls on extendibility

Basic Coding Pattern

```
EventMaskType myMask;
while (1)
{
    WaitEvent(evDropBall|evLooseBall|evReachedFirePosition|
    evPullRubber|evFire|evReload|evReachedLowPosition|evReadyToFire);
    GetEvent(Control, (EventMaskRefType) &myMask);
    ClearEvent(myMask)

    switch (myState)
    {
        case ISREADY: if (myMask & evDropBall)
            {
                myState = ISMOVINGUP;
                else if (second event...)
                    break;

                ...
            } //switch
    } //while
```

States

Event triggering a transition leaving the state

Resulting state

Extended Coding Pattern

State Action Expressions

```
switch (myState)
{
    case ISREADY:
        if (myMask & evDropBall)
        {
            //Exit action of state ISREADY

            //Transition Action
            SetMoveGunUp();

            //State Change
            myState = ISMOVINGUP;

            //Entry action for state ISMOVINGUP
        }
        break;

    ...
} //switch
} //while
```

Transition
expression

State / Exit actions

State / Entry
actions
(distributed!)

State / Do actions should not be used, as they will not be called in a deterministic way.

Extended Coding Pattern

Guard Conditions

```
switch (myState)
{
    case ISREADY: //Do action of state ISREADY
        if ((myMask & evDropBall) && (MachineHasHalted())
        {
            //Exit action of state ISREADY

            //Transition Action
            SetMoveGunUp();

            //State change
            myState = ISMOVINGUP;

            //Entry action for state ISMOVINGUP

        }
        break;

    ...
} //switch
} //while
```

Guard Condition



OSEK Limitations

Commands which effect the scheduler, like `WaitEvent`, `TerminateTask`, `Schedule...` may not be mixed up with ressources.

```
GetRessource (ressource) ;  
WaitEvent (myEventMask) ;  
  
//Some more code
```

will lead to unexpected behaviour. Correct order:

```
WaitEvent (myEventMask) ;  
GetRessource (ressource) ;
```

8.2 Restrictions when using resources

TerminateTask, *ChainTask*, *Schedule*, *WaitEvent* shall not be called while a resource is occupied. Interrupt service routine shall not be completed with a resource occupied.

In case of multiple resource occupation within one task, the user has to request and release resources following the LIFO principle (stack like).

Source: OSEK specification

OSEK Limitations

If a task is terminated, received events will be deleted.

- Do not use cyclic tasks to receive events
- Implement WaitEvent inside an endless loop

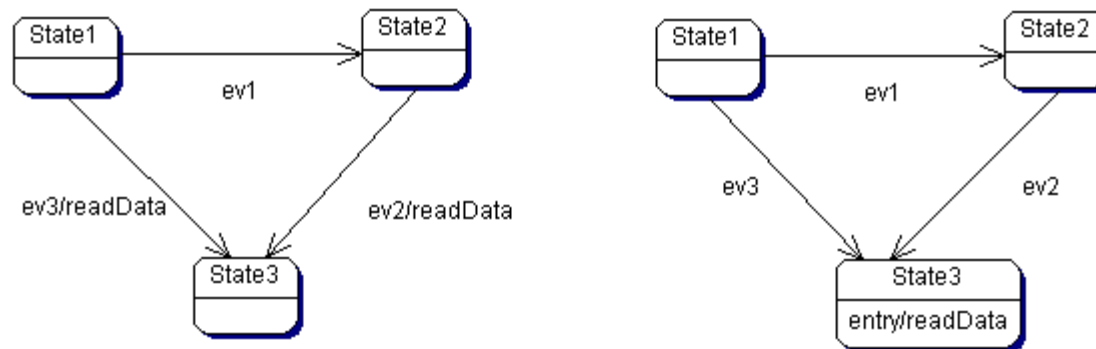
```
//Inside InitTask
ActivateTask(Control);

Task(Control)
while (1) {
    WaitEvent (myEventMask);
    //Process event
}
```

Event Actions and State Actions

Transition and State Actions can be transformed, however

- Transition expressions are recommended if the activity is related to the transition only
- State expressions are recommended if the activity is related to the state (i.e. are valid for all transitions)



MP3 Player

- A simulation of a MP3 Player shall be developed
- Output
 - Volume
 - Remaining time of the song
 - Title of the song
 - Menu
- The MP3 Player has 4 buttons: Up, Down, Stop, Play
- The MP3 Player has 2 modes: Standby, Play
- The buttons initiate different actions depending on the mode

■ Up/Down	Standby: Select Song	Play: Volume
■ Stop	Standby: -	Play: Stop Song, Change to Standby
■ Play	Standby: Start Song Change to Play mode	Play: -



Let's go

Identify the required

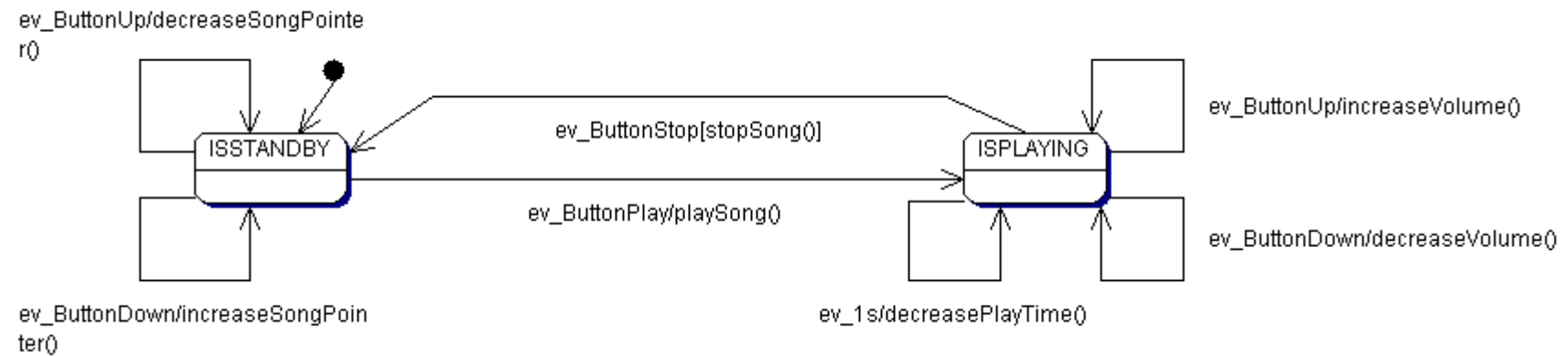
- States
- Events
- Activities
- Guard Conditions

Design the State Machine

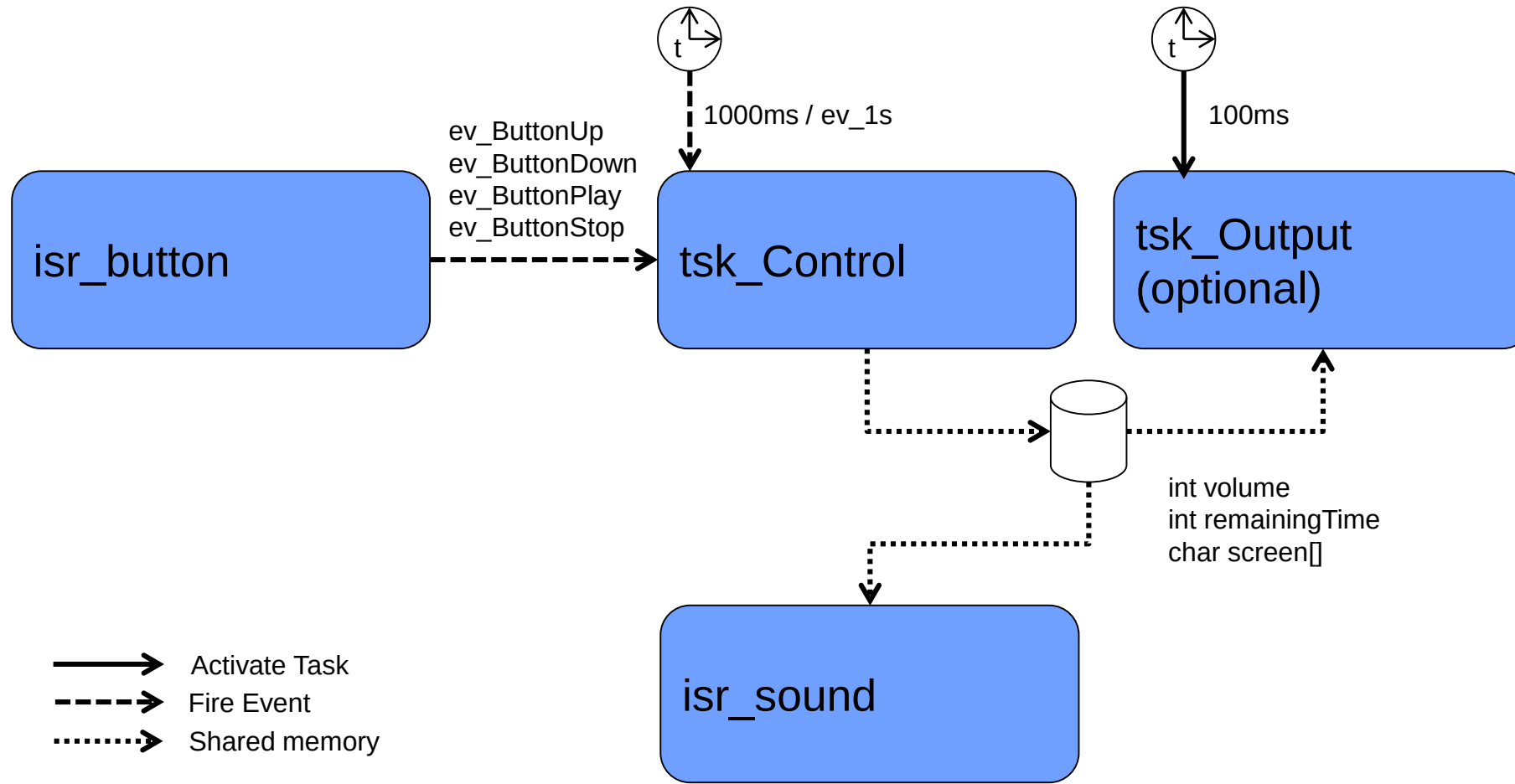
- Which tasks are required?
- How many Counters and Alarms are needed?



Task Machine MP3 Player



Task Model MP3 Player



Recipe

Iteration 1

- Create state machine design incl. clear states, events, actions and guards
- Configure OS
- Create Tasks, Activation Code
- Create empty `statemachine.run(event)` method / function
- Add `log(event, state)` function to the empty statemachine
- Test – Check if all events are fired

Iteration 2

- Add empty actions and guards
- Implement the state machine logic
- Test – Check if all transitions are correctly implemented

Iteration 3...n

- Add the code to the actions and guards
- Test the behaviour

Wrapup

- Motivation State Machines
- Basic Pattern - Switch Case Implementation